PYTHON UNIT I — RECHECKED CODING QUESTION BANK

140 Coding Questions + Complete Python Solutions • Beginner → Medium → Higher Logic • Question first, solution immediately below

Purpose: This revised version was rechecked against the syllabus represented in the supplied course PDF. It strengthens beginner understanding and adds more important coding practice.

SYLLABUS COVERAGE CHECK

• Directly represented from the supplied material: Python basics, variables, data types, Boolean logic, arithmetic/comparison/logical operators, assignment operators, identity operators, membership operators, bitwise operators, walrus operator, and conditional statements.

• Beginner reinforcement added: input(), type conversion, indexing, mutable vs immutable examples, dictionary operations, truthiness, operator precedence, short-circuit logic, and tracing.

• Syllabus-listed practice areas also retained: for/while loops, range(), lists, text files, Pandas, NumPy, and Matplotlib.

• Important logic-building programs added: factorial, prime, primes in a range, palindrome, Armstrong number, reverse number, digit sum, multiplication table, largest/second-largest, frequency dictionary, duplicate removal, common elements, and calculator.

• Important note: the supplied 60-page material lists some later topics in the syllabus without teaching them in the same depth; those are labeled/treated here as syllabus practice rather than falsely presented as detailed source-derived theory.

A. Python Basics, Variables & Data Types

Question 1

Write a Python program to print your name and course.

```
name = "Charan"
course = "B.Tech Cyber Security"
print(name)
print(course)
```

Why this is important: Practice print(), strings, and simple variables.

Question 2

Store an integer in a variable and print its value and type.

```
x = 25
print(x)
print(type(x))
```

Why this is important: An integer has type int.

Question 3

Store a decimal number and display its type.

```
x = 12.5
print(x)
print(type(x))
```

Why this is important: A decimal value is normally a float.

Question 4

Create a complex number and display its type.

```
z = 3 + 4j
print(z)
print(type(z))
```

Why this is important: Complex numbers use real and imaginary parts.

Question 5

Create a string and print its first character.

```
text = "PYTHON"
print(text[0])
```

Why this is important: String indexing starts at 0.

Question 6

Create a list and print its first and last elements.

```
items = [10, 20, 30, 40]
print(items[0])
print(items[-1])
```

Why this is important: Lists support indexing; -1 means the last element.

Question 7

Create a tuple and print its second element.

```
t = (5, 10, 15)
print(t[1])
```

Why this is important: Tuple indexing also starts at 0.

Question 8

Create a Boolean variable and print its type.

```
flag = True
print(flag)
print(type(flag))
```

Why this is important: Boolean values are True and False.

Question 9

Create a dictionary containing a student's name and mark, then print the name.

```
student = {"name": "Charan", "mark": 85}
print(student["name"])
```

Why this is important: Dictionary values are accessed using their keys.

Question 10

Swap two variables without using a third variable.

```
a = 10
b = 20
a, b = b, a
print(a, b)
```

Why this is important: Python supports multiple assignment for swapping.

Question 11

Assign three values to three variables in one statement.

```
a, b, c = 10, 20, 30
print(a, b, c)
```

Why this is important: This is multiple assignment.

Question 12

Assign the same value to three variables.

```
a = b = c = 100
print(a, b, c)
```

Why this is important: Chained assignment gives the same value to multiple variables.

Question 13

Read two numbers from the user and print their sum.

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
print("Sum =", a + b)
```

Why this is important: input() returns text, so int() converts it to an integer.

Question 14

Read two numbers and print sum, difference, product, and quotient.

```
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
print("Sum =", a + b)
print("Difference =", a - b)
print("Product =", a * b)
print("Quotient =", a / b)
```

Why this is important: This combines basic arithmetic operators.

Question 15

Find the remainder when one number is divided by another.

```
a = 17
b = 5
print(a % b)
```

Why this is important: The % operator gives the remainder.

Question 16

Calculate a number raised to a power.

```
base = 2
power = 5
print(base ** power)
```

*Why this is important: The ** operator performs exponentiation.*

Question 17

Demonstrate floor division.

```
a = 17
b = 5
print(a // b)
```

Why this is important: // gives the floor-divided integer result.

Question 18

Increase a variable by 10 using a compound assignment operator.

```
x = 25
x += 10
print(x)
```

Why this is important: += updates the existing variable.

Question 19

Multiply a variable by 3 using a compound assignment operator.

```
x = 7
x *= 3
print(x)
```

*Why this is important: *= is a compound assignment operator.*

Question 20

Compare two values and print whether they are equal.

```
a = 10
b = 10
print(a == b)
```

Why this is important: == compares values for equality.

B. Operators, Boolean Logic & Conditions

Question 21

Check whether a number is greater than another number.

```
a = 15
b = 10
print(a > b)
```

Why this is important: The > operator checks greater-than.

Question 22
Check whether attendance is at least 75%.

```
attendance = 82
print(attendance >= 75)
```

Why this is important: >= includes equality.

Question 23
Check whether a number lies between 3 and 10 using and.

```
x = 7
print(x >= 3 and x <= 10)
```

Why this is important: and requires both conditions to be True.

Question 24
Check whether a number is less than 0 or greater than 100.

```
x = 120
print(x < 0 or x > 100)
```

Why this is important: or is True when at least one condition is True.

Question 25
Use not to reverse a Boolean condition.

```
x = 10
print(not (x > 20))
```

Why this is important: not changes True to False and False to True.

Question 26
Show the difference between == and is using two equal lists.

```
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b)
print(a is b)
```

Why this is important: == checks value equality; is checks object identity.

Question 27
Check whether an item exists in a list using in.

```
numbers = [10, 20, 30]
print(20 in numbers)
```

Why this is important: in tests membership.

Question 28
Check whether an item does not exist in a list.

```
numbers = [10, 20, 30]
print(50 not in numbers)
```

Why this is important: not in tests non-membership.

Question 29
Check whether a key exists in a dictionary.

```
student = {"name": "Charan", "mark": 90}
print("mark" in student)
```

Why this is important: Dictionary membership checks keys.

Question 30

Perform bitwise AND, OR, and XOR.

```
a = 6
b = 3
print(a & b)
print(a | b)
print(a ^ b)
```

Why this is important: These are bitwise operations on binary representations.

Question 31

Use bitwise NOT on a number.

```
x = 5
print(~x)
```

Why this is important: ~ performs bitwise complement.

Question 32

Demonstrate left and right bit shifts.

```
x = 8
print(x << 1)
print(x >> 1)
```

Why this is important: Shifting moves binary bits left or right.

Question 33

Use the walrus operator to assign and test a value.

```
if (n := 5) > 3:
print("n is greater than 3:", n)
```

Why this is important: The walrus operator := assigns a value inside an expression.

Question 34

Write an if statement that prints 'Pass' when mark is at least 40.

```
mark = 55
if mark >= 40:
print("Pass")
```

Why this is important: Basic if condition.

Question 35

Write an if-else program to check pass or fail.

```
mark = 35
if mark >= 40:
print("Pass")
else:
print("Fail")
```

Why this is important: if-else selects between two alternatives.

Question 36

Write an if-elif-else program for grades A, B, C, and Fail.

```
mark = 82
if mark >= 80:
print("A")
elif mark >= 60:
print("B")
elif mark >= 40:
print("C")
else:
print("Fail")
```

Why this is important: elif allows multiple conditions.

Question 37

Classify a temperature as hot, normal, or cold.

```
t = 32
if t >= 35:
print("Hot")
elif t >= 20:
print("Normal")
else:
print("Cold")
```

Why this is important: This practices ordered conditions.

Question 38

Use a nested if to check whether a student passed and then whether the mark is above 75.

```
mark = 82
if mark >= 40:
if mark > 75:
print("Pass with distinction")
else:
print("Pass")
else:
print("Fail")
```

Why this is important: A nested if is an if statement inside another if.

Question 39

Write a short-hand if statement that prints 'Adult' when age is 18 or above.

```
age = 20
if age >= 18: print("Adult")
```

Why this is important: This is the one-line form of if.

Question 40

Print the day name for a number from 1 to 7.

```
day = 3
if day == 1:
print("Monday")
elif day == 2:
print("Tuesday")
elif day == 3:
print("Wednesday")
elif day == 4:
print("Thursday")
elif day == 5:
print("Friday")
elif day == 6:
print("Saturday")
elif day == 7:
print("Sunday")
else:
print("Invalid day")
```

Why this is important: Combines equality comparison with if-elif-else.

Question 41

Check whether a number is positive, negative, or zero.

```
x = -4
if x > 0:
print("Positive")
elif x < 0:
print("Negative")
else:
print("Zero")
```

Why this is important: A common beginner decision problem.

Question 42

Check whether a number is even or odd.

```
n = 17
if n % 2 == 0:
print("Even")
else:
print("Odd")
```

Why this is important: The remainder after division by 2 identifies even/odd.

Question 43

Find the larger of two numbers using if-else.

```
a = 25
b = 18
if a > b:
print(a)
else:
print(b)
```

Why this is important: Comparison plus conditional execution.

Question 44

Find the largest of three numbers.

```
a, b, c = 12, 45, 27
if a >= b and a >= c:
print(a)
elif b >= a and b >= c:
print(b)
else:
print(c)
```

Why this is important: Combines comparisons and logical and.

Question 45

Check eligibility using both attendance and marks.

```
attendance = 80
mark = 55
if attendance >= 75 and mark >= 40:
print("Eligible")
else:
print("Not eligible")
```

Why this is important: Both conditions must be satisfied.

C. Data Structures & Beginner Logic

Question 46

Predict and print the Boolean result of a compound comparison.

```
x = 10
y = 20
print(x < y and y < 30)
```

Why this is important: Trace each comparison first, then apply and.

Question 47

Create a nested dictionary containing student details and print the city.

```
student = {
"name": "Charan",
"details": {"city": "Eluru", "year": 1}
}
print(student["details"]["city"])
```

Why this is important: Nested dictionaries use multiple keys.

Question 48

Add a new key-value pair to a dictionary.

```
student = {"name": "Charan"}
student["mark"] = 90
print(student)
```

Why this is important: Assigning a new key adds it.

Question 49

Delete a dictionary key using del.

```
student = {"name": "Charan", "mark": 90}
del student["mark"]
print(student)
```

Why this is important: del removes the specified key.

Question 50

Remove a dictionary key using pop() and print the removed value.

```
student = {"name": "Charan", "mark": 90}
removed = student.pop("mark")
print("Removed:", removed)
print(student)
```

Why this is important: pop() removes a key and returns its value.

Question 51

Convert a list to a tuple and print its second item.

```
items = [10, 20, 30]
t = tuple(items)
print(t[1])
```

Why this is important: tuple() converts the list to a tuple.

Question 52

Repeat a tuple three times.

```
t = (1, 2)
print(t * 3)
```

*Why this is important: Tuple repetition uses *.*

Question 53

Create a nested tuple and access an inner element.

```
t = ((1, 2), (3, 4))
print(t[1][0])
```

Why this is important: Nested indexing accesses the inner tuple.

Question 54

Demonstrate that a list can be changed after creation.

```
items = [10, 20, 30]
items[1] = 99
print(items)
```

Why this is important: Lists are mutable.

Question 55

Demonstrate string indexing from the beginning and end.

```
text = "PYTHON"
print(text[0])
print(text[-1])
```

Why this is important: Positive indexes count from the left; negative indexes count from the right.

Question 56

Check the truth value of None, an empty tuple, an empty dictionary, 0.0, and a non-empty string.

```
values = [None, (), {}, 0.0, "Python"]
for v in values:
print(repr(v), bool(v))
```

Why this is important: Python treats several empty/zero-like values as False and a non-empty string as True.

Question 57

Create a new string by adding text to an existing string.

```
text = "Hello"
text = text + " Python"
print(text)
```

Why this is important: Strings cannot be changed in place; a new string is created.

Question 58

Show what happens when a dictionary contains the same key twice.

```
d = {"a": 10, "a": 20}
print(d)
```

Why this is important: A dictionary keeps the later value for a duplicate key.

Question 59

Show that dictionary keys are case-sensitive.

```
d = {"Name": "Charan", "name": "Python"}
print(d["Name"])
print(d["name"])
```

Why this is important: Name and name are different keys.

Question 60

Show the difference between dictionary key membership and value membership.

```
d = {"name": "Charan", "mark": 90}
print("name" in d)
print("Charan" in d)
print("Charan" in d.values())
```

Why this is important: in on a dictionary checks keys; values can be checked with values().

Question 61

Evaluate an expression containing several arithmetic operators.

```
result = 10 + 5 * 2 - 8 / 4
print(result)
```

Why this is important: Python follows operator precedence.

Question 62

Trace a bitwise expression.

```
a = 5
b = 3
print(a & b)
print(a | b)
print(a ^ b)
```

Why this is important: Convert to binary mentally to understand each operation.

Question 63

Trace a sequence of compound assignments.

```
x = 10
x += 5
x *= 2
x -= 4
print(x)
```

Why this is important: Follow each update in order.

Question 64

Demonstrate short-circuit behavior with and.

```
x = 0
result = x != 0 and 10 / x > 1
print(result)
```

Why this is important: Because the first condition is False, the second part is not evaluated.

Question 65

Demonstrate short-circuit behavior with or.

```
x = 5
result = x > 0 or 10 / 0 > 1
print(result)
```

Why this is important: Because the first condition is True, the second part is not evaluated.

Question 66

Print numbers from 1 to 5 using range().

```
for i in range(1, 6):
print(i)
```

Why this is important: The stop value 6 is excluded.

Question 67

Print the first five natural numbers using range().

```
for i in range(1, 6):
print(i)
```

Why this is important: range(start, stop) generates values from start up to stop-1.

Question 68

Find the sum of numbers from 1 to 10 using a loop.

```
total = 0
for i in range(1, 11):
total += i
print(total)
```

Why this is important: This is an important beginner loop pattern.

Question 69

Print numbers from 1 to 5 using a while loop.

```
i = 1
while i <= 5:
print(i)
i += 1
```

Why this is important: The update i += 1 prevents an infinite loop.

Question 70

Find the factorial of a number using a for loop.

```
n = 5
fact = 1
for i in range(1, n + 1):
fact *= i
print(fact)
```

Why this is important: Factorial is a core beginner programming problem.

Question 71
Check whether a number is prime.

```
n = 29
if n < 2:
print("Not prime")
else:
prime = True
for i in range(2, n):
if n % i == 0:
prime = False
break
print("Prime" if prime else "Not prime")
```

Why this is important: A prime number has exactly two positive factors: 1 and itself.

Question 72
Print all prime numbers from 2 to 50.

```
for n in range(2, 51):
prime = True
for i in range(2, n):
if n % i == 0:
prime = False
break
if prime:
print(n)
```

Why this is important: Nested loops provide practice with repeated prime checking.

Question 73
Reverse a string using a loop.

```
text = "PYTHON"
rev = ""
for ch in text:
rev = ch + rev
print(rev)
```

Why this is important: Each character is placed at the beginning of the new string.

Question 74
Count the vowels in a string.

```
text = "education"
count = 0
for ch in text.lower():
if ch in "aeiou":
count += 1
print(count)
```

Why this is important: Combines a loop, membership, and counter.

Question 75
Find the largest element in a list without using max().

```
numbers = [12, 45, 7, 89, 23]
largest = numbers[0]
for n in numbers:
if n > largest:
largest = n
print(largest)
```

Why this is important: Initialize with the first element, then compare each item.

D. Loops, Lists, Files & Core Programs

Question 76

Find the second-largest distinct value in a list.

```
numbers = [10, 50, 30, 50, 20]
unique = list(set(numbers))
unique.sort()
print(unique[-2])
```

Why this is important: The set removes duplicates before sorting.

Question 77

Insert, replace, and remove an element from a list.

```
items = [10, 20, 30]
items.insert(1, 15)
items[0] = 5
items.remove(30)
print(items)
```

Why this is important: These are important basic list operations.

Question 78

Search for an item in a list and print whether it exists.

```
items = [10, 20, 30, 40]
target = 30
if target in items:
print("Found")
else:
print("Not found")
```

Why this is important: Uses list membership with in.

Question 79

Sort a list in ascending order.

```
numbers = [5, 2, 9, 1, 7]
numbers.sort()
print(numbers)
```

Why this is important: sort() changes the list into ascending order by default.

Question 80

Traverse a dictionary and print each key and value.

```
student = {"name": "Charan", "mark": 90}
for key, value in student.items():
print(key, value)
```

Why this is important: items() gives key-value pairs.

Question 81

Create a frequency dictionary for characters in a string.

```
text = "banana"
freq = {}
for ch in text:
freq[ch] = freq.get(ch, 0) + 1
print(freq)
```

Why this is important: This is a useful real-world dictionary pattern.

Question 82
Read a text file and print its contents.

```
with open("sample.txt", "r") as f:
print(f.read())
```

Why this is important: open(..., 'r') reads a text file.

Question 83
Write text into a file.

```
with open("sample.txt", "w") as f:
f.write("Python is easy to learn.")
```

Why this is important: w mode creates/overwrites the file.

Question 84
Append a new line to a text file.

```
with open("sample.txt", "a") as f:
f.write("\nKeep practicing.")
```

Why this is important: a mode adds content at the end.

Question 85
Create a NumPy array and calculate its sum.

```
import numpy as np
a = np.array([10, 20, 30])
print(a.sum())
```

Why this is important: NumPy is included in the syllabus as a practice area.

Question 86
Create a 2x2 NumPy array and print it.

```
import numpy as np
a = np.array([[1, 2], [3, 4]])
print(a)
```

Why this is important: A 2D NumPy array can represent a matrix.

Question 87
Create a Pandas DataFrame from a dictionary.

```
import pandas as pd
data = {"Name": ["A", "B"], "Mark": [80, 90]}
df = pd.DataFrame(data)
print(df)
```

Why this is important: Pandas DataFrame is a common tabular data structure.

Question 88
Select one column from a Pandas DataFrame.

```
import pandas as pd
df = pd.DataFrame({"Name": ["A", "B"], "Mark": [80, 90]})
print(df["Mark"])
```

Why this is important: A column can be selected by its name.

Question 89

Create a simple Matplotlib line plot.

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4]
y = [2, 4, 6, 8]
plt.plot(x, y)
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Simple Line Plot")
plt.show()
```

Why this is important: Matplotlib is used for plotting and visualization.

Question 90

Create a dictionary containing the squares of even numbers from 1 to 10.

```
squares = {}
for n in range(1, 11):
if n % 2 == 0:
squares[n] = n * n
print(squares)
```

Why this is important: Combines range, condition, arithmetic, and dictionaries.

Question 91

Print numbers from 1 to 100 divisible by both 3 and 5.

```
for n in range(1, 101):
if n % 3 == 0 and n % 5 == 0:
print(n)
```

Why this is important: Combines loops, modulo, and logical and.

Question 92

Check whether a string is a palindrome.

```
text = "madam"
if text == text[::-1]:
print("Palindrome")
else:
print("Not palindrome")
```

Why this is important: A palindrome reads the same forward and backward.

Question 93

Find the sum of digits of an integer.

```
n = 5832
total = 0
while n > 0:
total += n % 10
n //= 10
print(total)
```

Why this is important: Uses modulo and floor division to process digits.

Question 94

Reverse an integer without converting it to a string.

```
n = 12345
rev = 0
while n > 0:
digit = n % 10
rev = rev * 10 + digit
n //= 10
print(rev)
```

Why this is important: This is an important logic-building problem.

Question 95

Check whether a number is an Armstrong number.

```
n = 153
temp = n
digits = len(str(n))
total = 0
while temp > 0:
digit = temp % 10
total += digit ** digits
temp //= 10
print("Armstrong" if total == n else "Not Armstrong")
```

Why this is important: Combines loops, digits, exponentiation, and conditions.

Question 96

*Build a menu-style calculator for +, -, *, and /.*

```
a = 20
b = 5
op = "*"

if op == "+":
print(a + b)
elif op == "-":
print(a - b)
elif op == "*":
print(a * b)
elif op == "/":
print(a / b)
else:
print("Invalid operator")
```

Why this is important: A good beginner problem combining operators and conditional logic.

Question 97

Print the multiplication table of a number.

```
n = 7
for i in range(1, 11):
print(n, "x", i, "=", n * i)
```

Why this is important: A common loop and arithmetic exercise.

Question 98

Count even and odd numbers in a list.

```
numbers = [1, 2, 3, 4, 5, 6]
even = odd = 0
for n in numbers:
if n % 2 == 0:
even += 1
else:
odd += 1
print("Even:", even)
print("Odd:", odd)
```

Why this is important: Combines list traversal, modulo, and counters.

Question 99

Remove duplicates from a list while preserving the original order.

```
numbers = [3, 1, 3, 2, 1, 4]
result = []
for n in numbers:
if n not in result:
result.append(n)
print(result)
```

Why this is important: Practices list membership and controlled insertion.

Question 100

Find common elements between two lists.

```
a = [1, 2, 3, 4]
b = [3, 4, 5, 6]
common = []
for x in a:
if x in b:
common.append(x)
print(common)
```

Why this is important: Combines loops, lists, and membership.

E. NumPy, Pandas & Matplotlib — Syllabus Practice

Question 101

Use or with a short-circuit expression.

```
age = 20
result = age < 0 or age >= 18
print(result)
```

Why this is important: or becomes True when at least one condition is True.

Question 102

Check whether a student can attend an exam using attendance and fee status.

```
attendance = 80
fee_paid = True
if attendance >= 75 and fee_paid:
print("Allowed")
else:
print("Not allowed")
```

Why this is important: A realistic beginner decision problem using and.

Question 103

Use a nested condition to classify a number as positive-even, positive-odd, or non-positive.

```
n = 14
if n > 0:
if n % 2 == 0:
print("Positive even")
else:
print("Positive odd")
else:
print("Non-positive")
```

Why this is important: Combines nested if with modulo.

Question 104

Write a program that demonstrates the difference between / and //.

```
a = 17
b = 5
print("True division:", a / b)
print("Floor division:", a // b)
```

Why this is important: This is a frequently confused operator pair.

Question 105

Write a program that demonstrates ==, !=, >, <, >=, and <=.

```
a = 10
b = 20
print(a == b)
print(a != b)
print(a > b)
print(a < b)
print(a >= b)
print(a <= b)
```

Why this is important: Practice all major comparison operators together.

Question 106

Use membership operators with both a list and a string.

```
numbers = [10, 20, 30]
text = "python"
print(20 in numbers)
print(50 not in numbers)
print("py" in text)
```

Why this is important: Membership works with several Python collection types.

Question 107

Create one program that demonstrates arithmetic, comparison, logical, identity, membership, and bitwise operators.

```
a = 10
b = 3
print(a + b)
print(a > b)
print(a > 5 and b < 5)
x = [1, 2]
y = x
print(x is y)
print(2 in x)
print(a & b)
```

Why this is important: This is a high-value revision program covering the main operator families.

FINAL BEGINNER REVISION CHECKLIST

☐ *Know print(), input(), variables, and basic type conversion.*

☐ *Know int, float, complex, str, list, tuple, dict, bool.*

☐ *Practice indexing and simple collection access.*

☐ *Know arithmetic, comparison, logical, assignment, identity, membership, and bitwise operators.*

☐ *Understand == versus is.*

☐ *Understand in versus not in, especially with dictionaries.*

☐ *Understand operator precedence and short-circuit and/or.*

☐ *Practice if, if-else, if-elif-else, nested if, and one-line if.*

☐ *Practice for, while, range(), counters, accumulators, and break.*

☐ *Practice lists, tuples, and dictionaries.*

☐ *Practice text-file read/write/append.*

☐ *Revise prime, factorial, palindrome, Armstrong, reverse number, digit sum, largest, second-largest, frequency, duplicates, and common elements.*

☐ *Review the syllabus-listed NumPy, Pandas, and Matplotlib practice questions.*

CONTINUATION — QUESTIONS 108 TO 140

These 33 questions complete the revised 140-question bank. They strengthen beginner understanding and cover important exam-style logic.

Question 108

Read a number from the user and check whether it is even or odd.

```
n = int(input("Enter a number: "))
if n % 2 == 0:
print("Even")
else:
print("Odd")
```

Why this is important: Input, conversion, condition, and output.

Question 109

Read a number and print its square and cube.

```
n = int(input("Enter a number: "))
print("Square =", n ** 2)
print("Cube =", n ** 3)
```

Why this is important: Input and exponentiation.

Question 110

Read three numbers and calculate their average.

```
a = float(input("A: "))
b = float(input("B: "))
c = float(input("C: "))
average = (a + b + c) / 3
print("Average =", average)
```

Why this is important: Arithmetic and grouping.

Question 111

Check whether a number is divisible by 2, 3, both, or neither.

```
n = 12
if n % 2 == 0 and n % 3 == 0:
print("Divisible by both")
elif n % 2 == 0:
print("Divisible by 2")
elif n % 3 == 0:
print("Divisible by 3")
else:
print("Neither")
```

Why this is important: Modulo, logical and, and elif.

Question 112

Find the smallest of three numbers.

```
a, b, c = 12, 4, 19
if a <= b and a <= c:
print(a)
elif b <= a and b <= c:
print(b)
else:
print(c)
```

Why this is important: Comparison logic.

Question 113

Print all even numbers from 1 to 20 using range().

```
for i in range(1, 21):
if i % 2 == 0:
print(i)
```

Why this is important: Loop plus condition.

Question 114
Print all odd numbers from 1 to 20 using range().

```
for i in range(1, 21):
if i % 2 != 0:
print(i)
```

Why this is important: Modulo and comparison.

Question 115
Find the sum of all even numbers from 1 to 50.

```
total = 0
for i in range(1, 51):
if i % 2 == 0:
total += i
print(total)
```

Why this is important: Accumulator pattern.

Question 116
Count how many numbers from 1 to 100 are divisible by 7.

```
count = 0
for i in range(1, 101):
if i % 7 == 0:
count += 1
print(count)
```

Why this is important: Counting pattern.

Question 117
Find the sum of elements in a list without using sum().

```
numbers = [10, 20, 30, 40]
total = 0
for n in numbers:
total += n
print(total)
```

Why this is important: List traversal and accumulator.

Question 118
Find the average of elements in a list.

```
numbers = [10, 20, 30, 40]
total = 0
for n in numbers:
total += n
average = total / len(numbers)
print(average)
```

Why this is important: List traversal and len().

Question 119
Count how many times a target appears in a list.

```
numbers = [2, 5, 2, 7, 2, 9]
target = 2
count = 0
for n in numbers:
if n == target:
count += 1
print(count)
```

Why this is important: List search and counter.

Question 120
Find the smallest element in a list without using min().

```
numbers = [8, 3, 12, 1, 9]
smallest = numbers[0]
for n in numbers:
if n < smallest:
smallest = n
print(smallest)
```

Why this is important: Core list logic.

Question 121
Copy a list and show that changing the copy does not change the original using copy().

```
a = [1, 2, 3]
b = a.copy()
b[0] = 99
print("Original:", a)
print("Copy:", b)
```

Why this is important: Beginner understanding of list copying.

Question 122
Join a list of words into one string.

```
words = ["Python", "is", "easy"]
sentence = " ".join(words)
print(sentence)
```

Why this is important: Useful string/list operation.

Question 123
Count the number of words in a sentence.

```
sentence = "Python is easy to learn"
words = sentence.split()
print(len(words))
```

Why this is important: Text processing and lists.

Question 124
Create a dictionary from two lists of keys and values.

```
keys = ["name", "mark"]
values = ["Charan", 90]
d = dict(zip(keys, values))
print(d)
```

Why this is important: Dictionary construction.

Question 125
Find the key having the largest value in a dictionary.

```
marks = {"A": 75, "B": 92, "C": 81}
best = max(marks, key=marks.get)
print(best)
```

Why this is important: Practical dictionary pattern.

Question 126
Count vowels and consonants in a string.

```
text = "python"
vowels = consonants = 0
for ch in text.lower():
if ch in "aeiou":
vowels += 1
elif ch.isalpha():
consonants += 1
print("Vowels:", vowels)
print("Consonants:", consonants)
```

Why this is important: Membership, conditions, and counters.

Question 127
Check whether a number is a perfect number.

```
n = 28
total = 0
for i in range(1, n):
if n % i == 0:
total += i
print("Perfect" if total == n else "Not perfect")
```

Why this is important: Higher-level beginner logic.

Question 128
Print the factors of a number.

```
n = 24
for i in range(1, n + 1):
if n % i == 0:
print(i)
```

Why this is important: Modulo and loops.

Question 129
Count the factors of a number.

```
n = 24
count = 0
for i in range(1, n + 1):
if n % i == 0:
count += 1
print(count)
```

Why this is important: Factor counting.

Question 130
Check whether a number is a perfect square using a simple loop.

```
n = 49
found = False
for i in range(1, n + 1):
if i * i == n:
found = True
break
print("Perfect square" if found else "Not a perfect square")
```

Why this is important: Loop, condition, Boolean flag, and break.

Question 131
Print the first 10 multiples of a number using a while loop.

```
n = 6
i = 1
while i <= 10:
print(n * i)
i += 1
```

Why this is important: While-loop control.

Question 132
Use break to stop a loop when a number greater than 50 is found.

```
numbers = [10, 20, 35, 60, 80]
for n in numbers:
if n > 50:
print("First value above 50:", n)
break
```

Why this is important: Important control-flow pattern.

Question 133
Use continue to print only odd numbers from 1 to 10.

```
for i in range(1, 11):
if i % 2 == 0:
continue
print(i)
```

Why this is important: Introduces continue.

Question 134
Read a text file line by line and print each line.

```
with open("sample.txt", "r") as f:
for line in f:
print(line.strip())
```

Why this is important: Text-file reading practice.

Question 135
Count the number of lines in a text file.

```
count = 0
with open("sample.txt", "r") as f:
for line in f:
count += 1
print("Lines:", count)
```

Why this is important: File reading with a counter.

Question 136
Create a NumPy array and print its shape.

```
import numpy as np
a = np.array([[1, 2, 3], [4, 5, 6]])
print(a.shape)
```

Why this is important: Basic NumPy array property.

Question 137
Create a Pandas DataFrame and display rows where marks are at least 80.

```
import pandas as pd
df = pd.DataFrame({"Name": ["A", "B", "C"], "Mark": [75, 85, 90]})
print(df[df["Mark"] >= 80])
```

Why this is important: Basic DataFrame filtering.

Question 138
Create a Matplotlib bar chart for three values.

```
import matplotlib.pyplot as plt
names = ["A", "B", "C"]
marks = [70, 85, 92]
plt.bar(names, marks)
plt.title("Marks")
plt.show()
```

Why this is important: Important visualization practice.

Question 139
Trace the output of a nested logical expression.

```
a = 10
b = 5
result = (a > b and b > 0) or (a == 0)
print(result)
```

Why this is important: Practice evaluating comparisons before and/or.

Question 140

Build one final revision program using input, operators, condition, loop, list, and dictionary.

```
n = int(input("Enter n: "))
numbers = []
for i in range(1, n + 1):
if i % 2 == 0:
numbers.append(i)

result = {
"count": len(numbers),
"sum": sum(numbers),
"largest": max(numbers) if numbers else None
}
print("Even numbers:", numbers)
print("Result:", result)
```

Why this is important: Final mixed revision of several core programming patterns.